

Intent Understanding

Intent Understanding

1. Course Content
2. Preparation
 - 2.1 Content Description
 - 2.2 Starting the Agent
 - 2.2 Configuring the Intent Mapping File
 - 2.3 Configuring the Knowledge Base
- 4.1 Starting the Program
- 4.2 Test Cases
5. Effect Debugging
 - 5.1 Visualized Workflow

1. Course Content

Basic: Master customizing unique user intent understanding functions through the RAG knowledge base.

Advanced: Master debugging the intent understanding effect on the Dify platform.

[!IMPORTANT]

- Intent understanding is designed to increase the rapport between the robot and the user, allowing the robot to understand the user more uniquely. This function should not be used to perform "strange" and "unconventional" tasks.

2. Preparation

2.1 Content Description

This section of the course uses the Jetson Orin NX as an example. For Raspberry Pi and Jetson-Nano boards, you need to open a terminal on the host machine, then enter the command to enter the Docker container. After entering the Docker container, enter the commands mentioned in this section of the course in the terminal. For instructions on entering the Docker container from the host machine, please refer to the content in [0. Instructions and Installation Steps] -> [Entering the Car's Docker (For Jetson-Nano and Raspberry Pi 5 users)] in this product tutorial. For Orin and NX boards, simply open the terminal and enter the commands mentioned in this section of the course.

2.2 Starting the Agent

Note: If it has already been started, there is no need to start it again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:

```
Jetson@yahboom: ~
Jetson@yahboom: ~ 177x26
Jetson@yahboom:~$ ./open_agent.sh
[1749538760.063253] Info | TermiosAgentLinux.cpp | Init | running... | fd: 3
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

2.2 Configuring the Intent Mapping File

- This file is used to store personal fuzzy intents and the corresponding tasks that the robot should perform. - Open the example file in the tutorial folder for this section. You can add multiple custom intents according to the reference format. Below is a simple example:

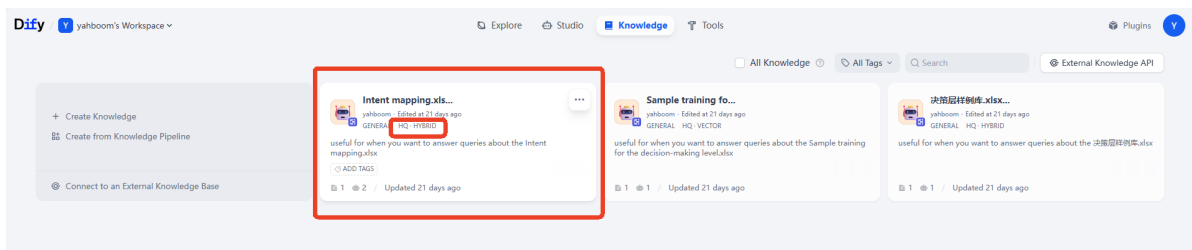
query	answer
I'm a little thirsty	1. Navigate to the kitchen, 2. Check if there is bottled water or drinks, 3. If there is, use the robotic arm to pick up the drink, 4. Navigate back to the starting position

2.3 Configuring the Knowledge Base

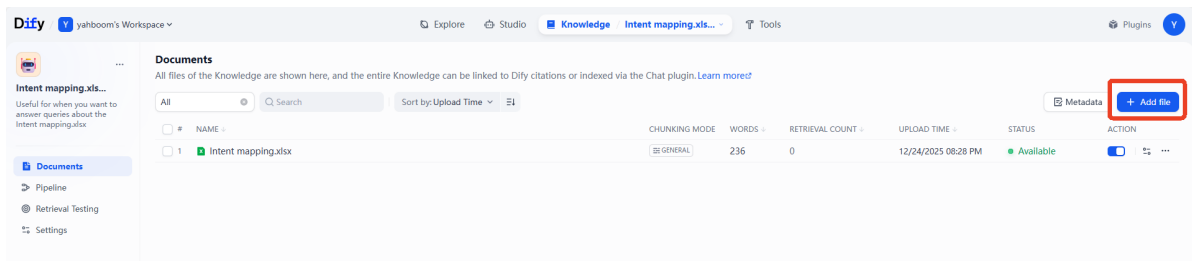
- Next, we need to upload the edited intent mapping file to Dify's RAG knowledge base.

[!TIP]

- For detailed instructions on using the RAG knowledge base, please refer to the tutorial in [2. AI Model Development - 06 - Deploy the RAG knowledge base].
- Dify comes pre-configured with a reference knowledge base for Intent mapping to demonstrate how to use the intent understanding function.



- You can modify the Intent mapping.xlsx template, or you can delete it and add your own file.



[!TIP]

Note that:

- To ensure the best intent understanding results, it is recommended to set the `Intent mapping` knowledge base to high-quality mode, as intent understanding often requires retrieving relevant snippets between similar semantic cues. ## 4. Running Examples

4.1 Starting the Program

- On the vehicle's onboard computer, open a terminal and enter the command to start the AI agent function:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

- Alternatively, you can use the shortcut command:

```
multi_brains
```

- On the vehicle's onboard computer, open two more terminals and enter the commands to start the navigation function:

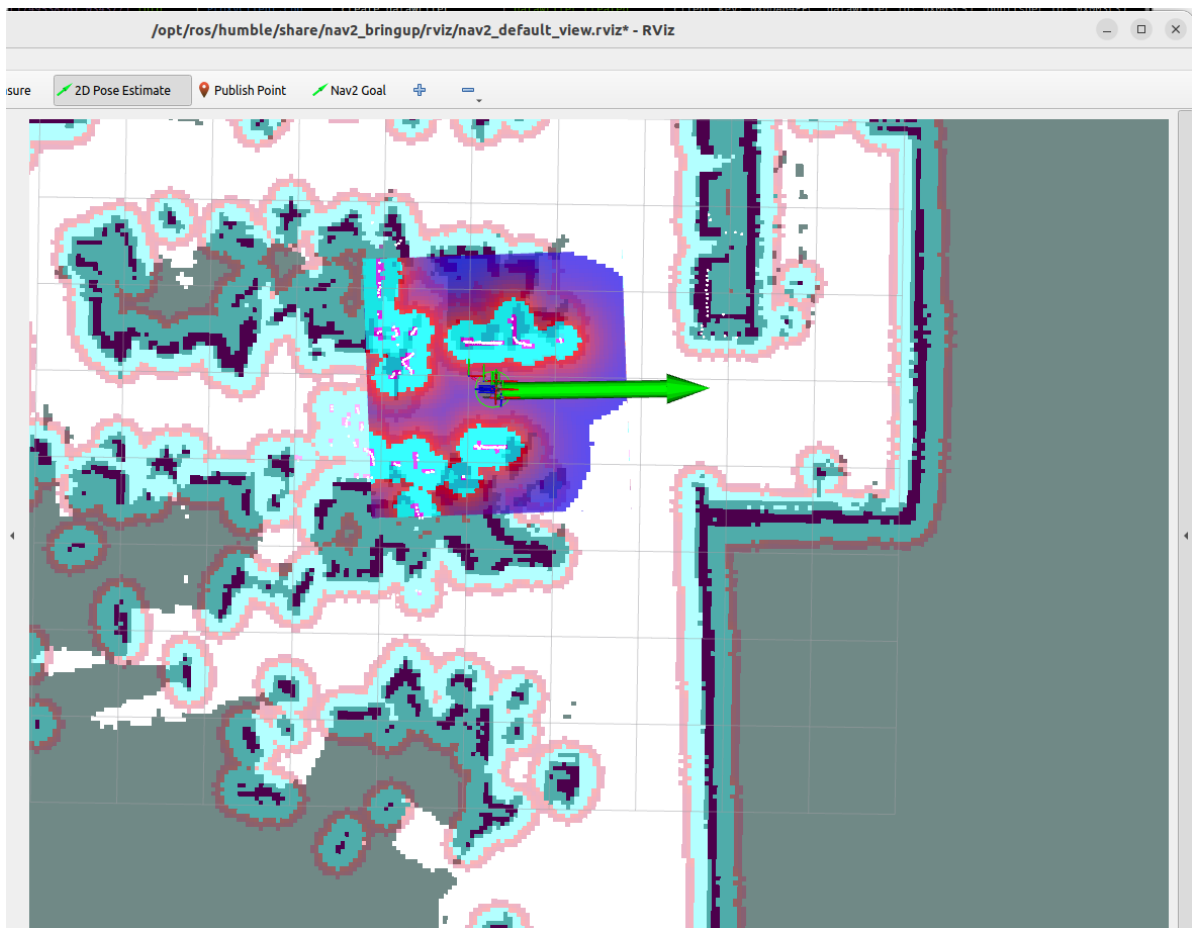
```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- On the robot, start rviz:

```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

Then, follow the procedure for starting the navigation function to initialize the positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate** in the toolbar at the top to enter the selection state. Roughly mark the robot's location and orientation on the map. After initializing the positioning, the preparation is complete.



4.2 Test Cases

These test cases are for reference only; users can create their own dialogue commands.

- I'm in the master bedroom now, and I feel a little thirsty.

```
asr-5] [INFO] [1750491006.444004139] [asr]: 0
asr-5] [INFO] [1750491006.534965377] [asr]: 0
asr-5] [INFO] [1750491006.624816840] [asr]: 0
asr-5] [INFO] [1750491006.715541268] [asr]: 0
asr-5] [INFO] [1750491006.805879567] [asr]: 0
asr-5] [INFO] [1750491006.897671209] [asr]: 0
asr-5] [INFO] [1750491006.989015568] [asr]: 0
asr-5] [INFO] [1750491007.079612532] [asr]: 0
asr-5] [INFO] [1750491007.171301831] [asr]: 0
asr-5] [INFO] [1750491007.263230564] [asr]: 0
asr-5] [INFO] [1750491007.354806162] [asr]: 0
asr-5] [INFO] [1750491007.445503515] [asr]: 0
asr-5] [INFO] [1750491007.536006330] [asr]: 0
asr-5] serial /dev/ttyUSB1 open
asr-5] funasr version: 1.2.6.
'load_data': '0.030', 'extract_feat': '0.021', 'forward': '1.870', 'batch_size': '1', 'rtf': '0.405'},
tf_avg: 0.405: 100%|██████████| 1/1 [00:01<00:00, 1.87s/it]
tf_avg: 0.405: 100%|██████████| 1/1 [00:01<00:00, 1.87s/it]
asr-5] [INFO] [1750491009.522065243] [asr]: 我现在在主卧室我感觉我有点口渴了
asr-5] [INFO] [1750491009.522748441] [asr]: okay😊, let me think for a moment...
model_service-3] [INFO] [1750491011.870241207] [model_service]: The upcoming task to be carried out: 1.
导航前往厨房
model_service-3] 2. 观察夹取矿泉水
model_service-3] 3. 导航返回主卧室
model_service-3] 4. 放下矿泉水
```

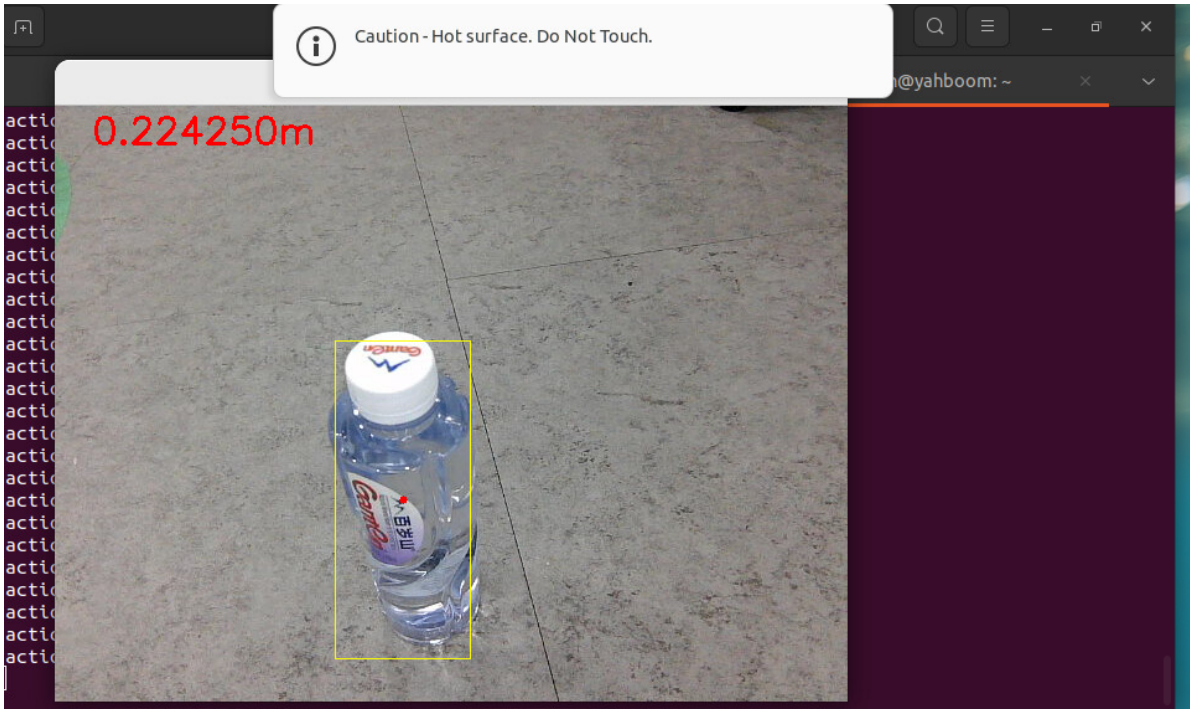
The task steps for the decision-making layer model planning are as follows:

```
model_service-3] [INFO] [1750491011.870241207] [model_service]: The upcoming task to be carried out: 1.
导航前往厨房
model_service-3] 2. 观察夹取矿泉水
model_service-3] 3. 导航返回主卧室
model_service-3] 4. 放下矿泉水
```

The tasks are executed sequentially according to the task steps planned by the decision-making layer's large model:


```
[model_service-3] [INFO] [1750491974.359941765] [model_service]: The upcoming task to be carried out: 导航前往厨房区域, 观察夹取矿泉水, 导航返回主卧室, 放下矿泉水。
[model_service-3] [INFO] [1750491977.376557382] [model_service]: "action": ['navigation(G)'], "response":
: 好的, 我这就去厨房帮你拿矿泉水。就像一个勇敢的骑士一样, 出发!
[action_service-4] [INFO] [1750491985.052591699] [action_service]: navigation Goal accepted
[action_service-4] [INFO] [1750492007.324887601] [action_service]: Navigation succeeded!
[model_service-3] [INFO] [1750492010.560577885] [model_service]: "action": ['seewhat()'], "response": 我已经到达厨房了, 现在开始寻找矿泉水。让我用我的火眼金睛看看它在哪里!
[model_service-3] [INFO] [1750492023.278365389] [model_service]: "action": ['grasp obj(305, 48, 392, 276)'], "response": 找到了! 矿泉水就在那里, 我这就去把它夹起来。看我的动作是不是很熟练?
[kin_srv-2] [WARN] [1750492032.926281990] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[action_service-4] start
[action_service-4] [INFO] [1750492036.744005351] [image_converter]: Received: [305, 48, 392, 276]
```

When the robotic arm is grasping, a visualization window will be displayed, as shown below:



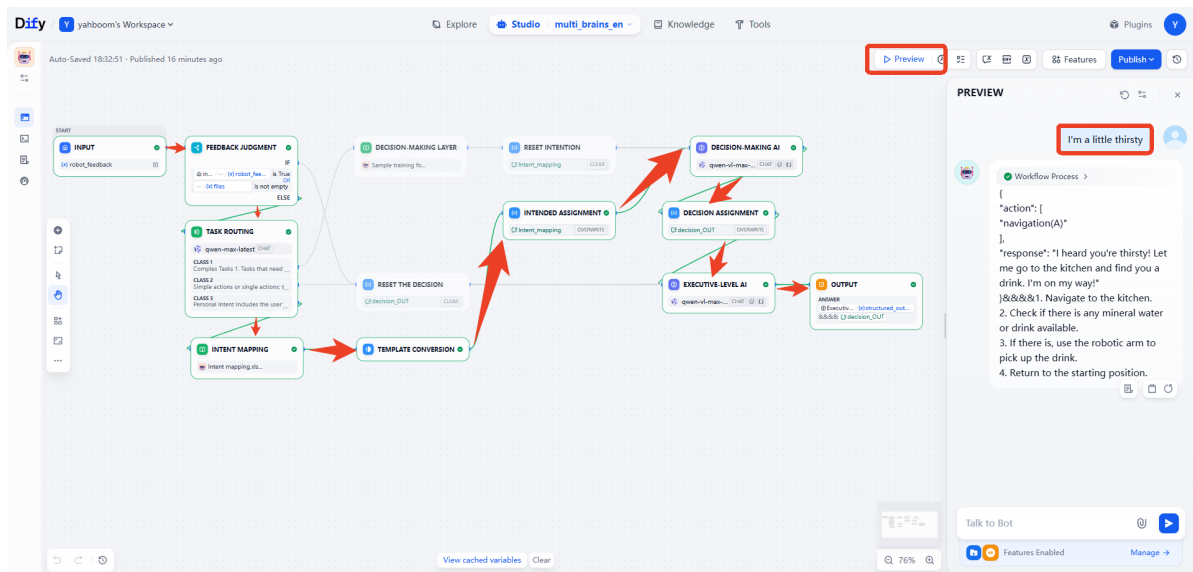
After arriving at the "master bedroom," the robot will use its robotic arm to put down the red block and prompt the user that the task is complete.

```
in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1750492056.766044511] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1750492056.774184364] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1750492056.781544546] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1750492056.789258466] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[kin_srv-2] [WARN] [1750492056.796697211] [kdl_parser]: The root link base_link has an inertia specified in the URDF, but KDL does not support a root link with an inertia. As a workaround, you can add an extra dummy link to your URDF.
[model_service-3] [INFO] [1750492065.414920973] [model_service]: "action": ['navigation(A)'], "response": 矿泉水已经稳稳地夹在手里了, 现在我准备返回主卧室。就像一个忠诚的信使, 使命必达!
[action_service-4] [INFO] [1750492074.955772124] [action_service]: navigation Goal accepted
[action_service-4] [INFO] [1750492092.422674757] [action_service]: Navigation succeeded!
[model_service-3] [INFO] [1750492095.280458636] [model_service]: "action": ['putdown()'], "response": 我已经顺利回到主卧室, 现在把矿泉水放在这里。请慢用, 希望这能解你的渴!
[model_service-3] [INFO] [1750492111.980638518] [model_service]: "action": ['finishtask()'], "response": 任务完成啦! 如果你还需要什么帮助, 随时告诉我哦。祝你有个美好的一天!
```

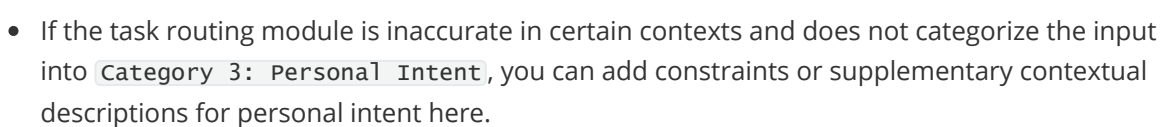
5. Effect Debugging

5.1 Visualized Workflow

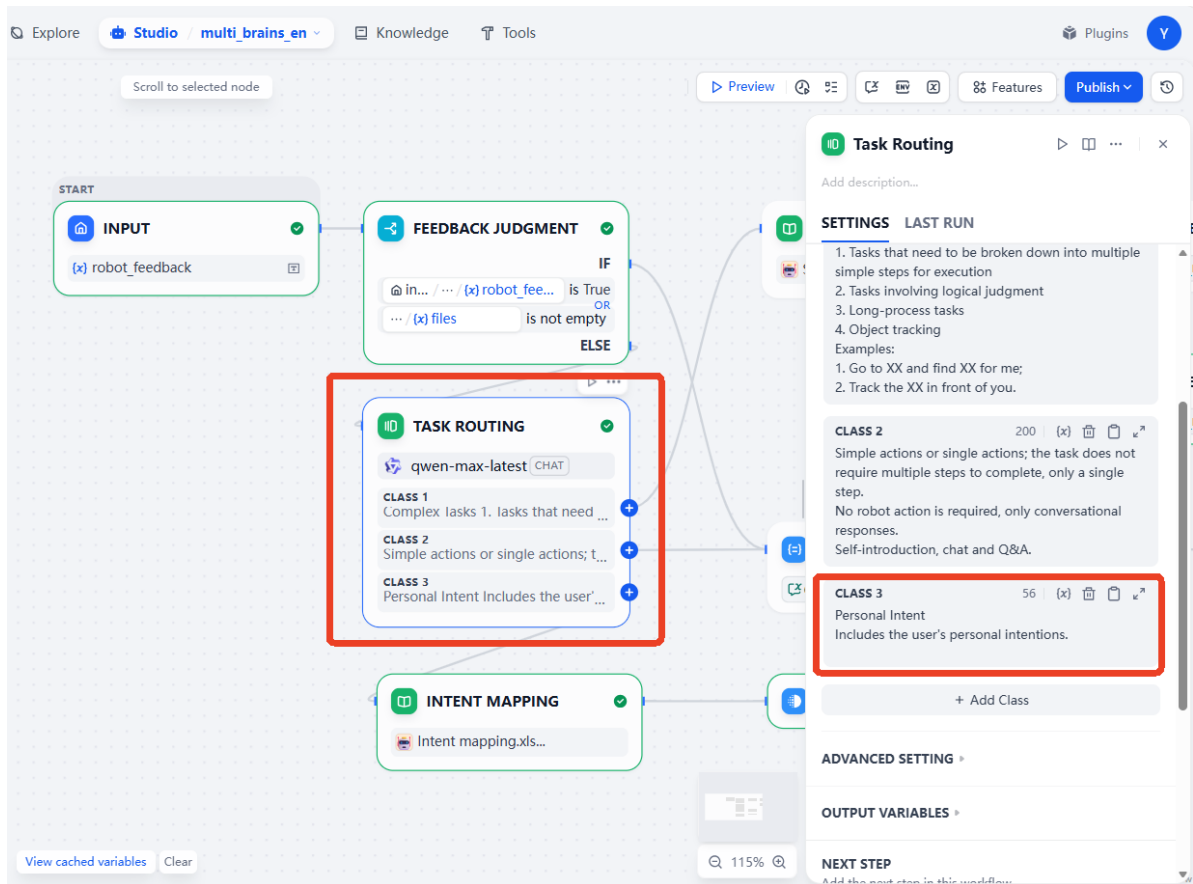
- Open the corresponding language version of the multi_brains application, then click preview to input test content and view the data flow.



- You can also open the workflow to view the time taken for each step and the input and output content of each process.



- If the task routing module is inaccurate in certain contexts and does not categorize the input into `Category 3: Personal Intent`, you can add constraints or supplementary contextual descriptions for personal intent here.



- In addition, the recall performance of input sentences in the knowledge base can also be tested separately within the intent mapping knowledge base.

The screenshot shows the Dify Knowledge base interface with a 'Retrieval Test' section and 'Retrieved Chunks'.

Retrieval Test:

- Intent mapping.xls...** (Useful for when you want to answer queries about the Intent mapping.xlsx)
- SOURCE TEXT:** I'm a little thirsty
- Test** button

Records:

QUERY CONTENT	SOURCE	TIME
I'm a little thirsty	Retrieval Test	01/14/2026 06:35 PM
I'm a little thirsty	App	01/14/2026 06:32 PM

1 Retrieved Chunks:

- Chunk-01 - 236 characters
- query: "I'm a little thirsty"; answer: "1. Navigate to the kitchen. 2. Check if there is any mineral water or drink available. 3. If there is, use the robotic arm to pick up the drink. 4. The navigation returns to the..."
- Score: 0.85
- Intent mapping.xlsx (OPEN)